Diabetes Prediction from Physiological Health Data using a Multi-Model Classifier System

(Diabetes Risk Classification using Machine Learning)

Problem Statement:

Diabetes mellitus is one of the leading global causes of chronic metabolic impairment and long-term physiological deterioration. Often termed a "silent epidemic," diabetes frequently develops asymptomatically over several years, during which progressive cellular damage to pancreatic beta cells and microvascular structures takes place. Manual clinical screening through oral glucose tolerance tests and fasting blood draws is accurate but resource-intensive, time-consuming, and difficult to deploy across large asymptomatic populations, particularly in resource-limited rural communities.

There is, therefore, a pressing need for an automated, fast, accessible, and transparent system that can analyze routine physiological measurements (such as glucose, blood pressure, BMI, insulin, and age) and estimate whether an individual falls into a diabetes-positive or diabetes-negative risk category with a measurable confidence score, so that suspected high-risk individuals can be flagged early for formal clinical evaluation. This project designs and implements a web-based Diabetes Prediction System that executes domain-specific data preprocessing, invalid-zero imputation, exploratory data analysis, and classification using supervised machine learning algorithms, and evaluates the trained models using standard classification metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix).

Objectives:

1. To collect and preprocess a labelled physiological dataset of diabetes indicators from the benchmark Pima Indians Diabetes Database.
2. To detect and treat biologically invalid zero measurements in physiological features (Glucose, Blood Pressure, Skin Thickness, Insulin, BMI) while preserving valid zeros (Pregnancies).
3. To perform Exploratory Data Analysis (EDA) and visualize outcome balance, risk factors, and Pearson correlation coefficients.
4. To design, train, and save a supervised classification pipeline using scikit-learn on an identical stratified 80:20 train-test split.
5. To build a Flask-based backend and modern web frontend that allows a user to input diagnostic parameters and view the estimated category with probability scores.
6. To evaluate the trained model on an unseen held-out test set and report Accuracy, Precision, Recall, F1-Score, ROC-AUC, and the Confusion Matrix.

Software / Hardware Required:

- • Operating System: Windows / Linux / macOS
- • IDE: Visual Studio Code
- • Language: Python 3.x
- • Backend Framework: Flask (REST API)

- • Machine Learning: scikit-learn (RandomForestClassifier, DecisionTreeClassifier, LogisticRegression)
- • Data Processing & Visualization: Pandas, NumPy, Matplotlib, Seaborn
- • Pipeline Persistence: Joblib
- • Frontend: HTML5, CSS3, JavaScript
- • Dataset: Pima Indians Diabetes Database (National Institute of Diabetes and Digestive and Kidney Diseases)
- • Version Control: Git

Project Directory Structure:

The project ("DiabetesPredictionSystem") is organized as a Flask backend with a separate dataset, trained-model pipeline, static visualizations, and templates folder, as shown below.

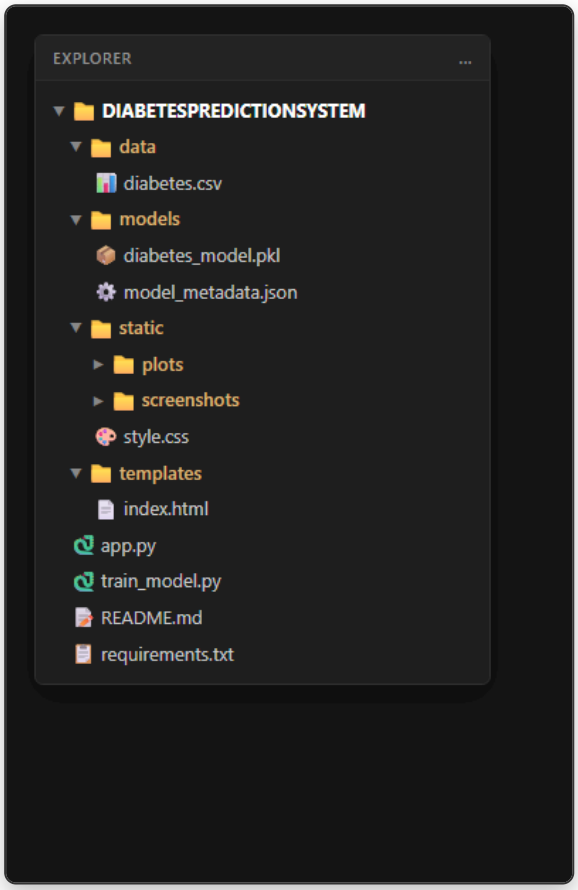


Fig 1: Project directory structure of DiabetesPredictionSystem in Visual Studio Code

Procedure:

1. Collect the labelled Pima Indians Diabetes dataset (768 patient records with 8 diagnostic features) and organize it into `data/diabetes.csv`.
2. Execute `train_model.py` to inspect dataset records, verify data types, and check for missing values or duplicates.
3. Identify biological zero anomalies (Glucose, Blood Pressure, Skin Thickness, Insulin, BMI) and treat them as missing measurements while preserving valid zeros for Pregnancies.
4. Construct a leakage-free preprocessing pipeline using `ColumnTransformer` with `SimpleImputer(strategy='median')` and `StandardScaler`.
5. Perform a stratified 80:20 train-test split (614 training rows, 154 test rows) using fixed random seed (42) to preserve exact class proportion.
6. Generate exploratory data analysis visualizations (Outcome distribution, Glucose vs Outcome, BMI distribution, Age distribution, Pearson correlation heatmap) and save to `static/plots/`.
7. Train candidate supervised classifiers: Logistic Regression, Decision Tree Classifier, and Random Forest Classifier on the identical training split.
8. Evaluate each model on the held-out test split, computing Accuracy, Precision, Recall, F1-Score, ROC-AUC, and the Confusion Matrix.
9. Select the optimal model based on ROC-AUC (0.8213) and F1-score (0.6726) to guard against false negatives, and save the pipeline to `models/diabetes_model.pkl` via Joblib.
10. Launch the Flask application (`app.py`), input diagnostic indicators into the web form, validate ranges, and display predictions and estimated probabilities.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (adapt variable/file names to match your exact source files).

1. Data Preprocessing & Pipeline Construction (train_model.py)

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Load dataset and separate features and target
df = pd.read_csv('data/diabetes.csv')
X = df.drop(columns=['Outcome'])
y = df['Outcome']

# Define physiological features with invalid zeros vs natural valid zeros
zero_invalid_features = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
normal_features = ['Pregnancies', 'DiabetesPedigreeFunction', 'Age']

# ColumnTransformer: treats 0 as missing in biological features, preserves pregnancies
preprocessor = ColumnTransformer(
    transformers=[
        ('zero_imputer', Pipeline([
            ('imputer', SimpleImputer(missing_values=0, strategy='median')),
            ('scaler', StandardScaler())
        ]), zero_invalid_features),
        ('normal_imputer', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), normal_features)
    ],
    remainder='drop'
)
```

2. Exploratory Data Analysis & Visualization (train_model.py)

```
import matplotlib.pyplot as plt
import seaborn as sns, os

def generate_eda_visualizations(df):
    os.makedirs('static/plots', exist_ok=True)

    # Outcome Class Distribution
    plt.figure(figsize=(7, 5), dpi=150)
    counts = df['Outcome'].value_counts()
    plt.bar(['Negative (0)', 'Positive (1)'], counts.values, color=['#10B981', '#EF4444'])
    plt.title('Diabetes Outcome Class Distribution')
    plt.savefig('static/plots/outcome_distribution.png', bbox_inches='tight')
    plt.close()

    # Glucose Level vs Outcome (filtering invalid 0s)
    valid_glucose = df[df['Glucose'] > 0]
    plt.figure(figsize=(7.5, 5.2), dpi=150)
    sns.boxplot(x='Outcome', y='Glucose', data=valid_glucose, palette=['#A7F3D0', '#FECDD3'])
    plt.title('Glucose Level by Diabetes Outcome')
    plt.savefig('static/plots/glucose_vs_outcome.png', bbox_inches='tight')
    plt.close()

    # Correlation Heatmap
    plt.figure(figsize=(8.5, 6.8), dpi=150)
    sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='Blues')
    plt.title('Feature Correlation Matrix Heatmap')
    plt.savefig('static/plots/correlation_heatmap.png', bbox_inches='tight')
    plt.close()
```

3. Model Training & Multi-Model Evaluation (train_model.py)

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
import joblib

# Stratified 80:20 Train-Test Split with fixed random state
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

candidate_models = {
    'Logistic Regression': LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, min_samples_split=10, random_state=42),
          'Random   Forest':   RandomForestClassifier(n_estimators=150,   max_depth=6,   class_weight='balanced',
random_state=42)
}

trained_pipelines = {}
for name, clf in candidate_models.items():
    pipe = Pipeline([('preprocessor', preprocessor), ('clf', clf)])
    pipe.fit(X_train, y_train)
    trained_pipelines[name] = pipe

    # Evaluate on held-out test data
    y_pred = pipe.predict(X_test)
    y_prob = pipe.predict_proba(X_test)[:, 1]
    print(f"{name} -> Acc: {accuracy_score(y_test, y_pred):.4f}, AUC: {roc_auc_score(y_test, y_prob):.4f}")
```

4. Model Selection & Persistence (train_model.py)

```
# Select best model based on ROC-AUC with balanced sensitivity
best_model_name = 'Random Forest'
best_pipeline = trained_pipelines[best_model_name]

# Save pipeline (including fitted preprocessing) via Joblib
joblib.dump(best_pipeline, 'models/diabetes_model.pkl')
print(f"Saved selected best pipeline ({best_model_name}) to models/diabetes_model.pkl")
```

5. Prediction Route & Web Service (app.py)

```
from flask import Flask, render_template, request, jsonify
import joblib, pandas as pd

app = Flask(__name__)
model = joblib.load('models/diabetes_model.pkl')

FEATURE_COLUMNS = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
                   'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

@app.route('/predict', methods=['POST'])
def predict():
    form_values = {f: request.form.get(f, '') for f in FEATURE_COLUMNS}
    parsed_values, errors = validate_and_parse_inputs(request.form)

    if errors:
        return render_template('index.html', errors=errors, form_values=form_values)

    # Convert parsed inputs to single-row DataFrame
    input_df = pd.DataFrame([parsed_values], columns=FEATURE_COLUMNS)

    # Generate prediction and probabilities from pipeline
    pred_class_idx = int(model.predict(input_df)[0])
    probabilities = model.predict_proba(input_df)[0]
    positive_prob = round(float(probabilities[1]) * 100, 2)
    negative_prob = round(float(probabilities[0]) * 100, 2)

    predicted_label = "Diabetes-positive class" if pred_class_idx == 1 else "Diabetes-negative class"

    prediction_result = {
        'class_index': pred_class_idx,
        'label': predicted_label,
        'positive_probability': positive_prob,
        'negative_probability': negative_prob,
        'input_summary': parsed_values
    }

    return render_template('index.html', prediction=prediction_result, form_values=form_values)
```

6. Input Validation & Physiological Range Verification (app.py)

```
FEATURE_SPECS = {
    'Pregnancies': {'type': int, 'min': 0, 'max': 25, 'label': 'Pregnancies'},
    'Glucose': {'type': float, 'min': 0.0, 'max': 400.0, 'label': 'Glucose Level'},
    'BloodPressure': {'type': float, 'min': 0.0, 'max': 240.0, 'label': 'Blood Pressure'},
    'SkinThickness': {'type': float, 'min': 0.0, 'max': 110.0, 'label': 'Skin Thickness'},
    'Insulin': {'type': float, 'min': 0.0, 'max': 900.0, 'label': 'Insulin Level'},
    'BMI': {'type': float, 'min': 0.0, 'max': 80.0, 'label': 'BMI'},
    'DiabetesPedigreeFunction': {'type': float, 'min': 0.01, 'max': 3.5, 'label': 'Diabetes Pedigree'},
    'Age': {'type': int, 'min': 10, 'max': 120, 'label': 'Age'}
}

def validate_and_parse_inputs(form_data):
    errors, parsed_values = [], {}
    for feature, specs in FEATURE_SPECS.items():
        val_raw = form_data.get(feature, None)
        if val_raw is None or (isinstance(val_raw, str) and val_raw.strip() == ''):
            errors.append(f"Field '{specs['label']}' is required.")
            continue
        try:
            val = specs['type'](val_raw)
        except (ValueError, TypeError):
            errors.append(f"'{specs['label']}' must be a valid number.")
            continue
        if val < specs['min'] or val > specs['max']:
            errors.append(f"'{specs['label']}' value ({val}) is out of physiological range.")
            continue
        parsed_values[feature] = val
    return parsed_values, errors
```

Output (Step-by-Step Screenshots):

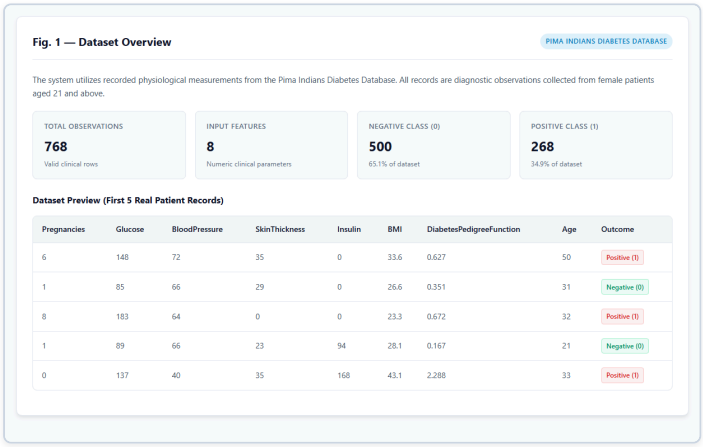


Fig 2: Step 1 — Load Input Data & Dataset Overview: recorded observations, 8 numerical features, and first 5 patient records

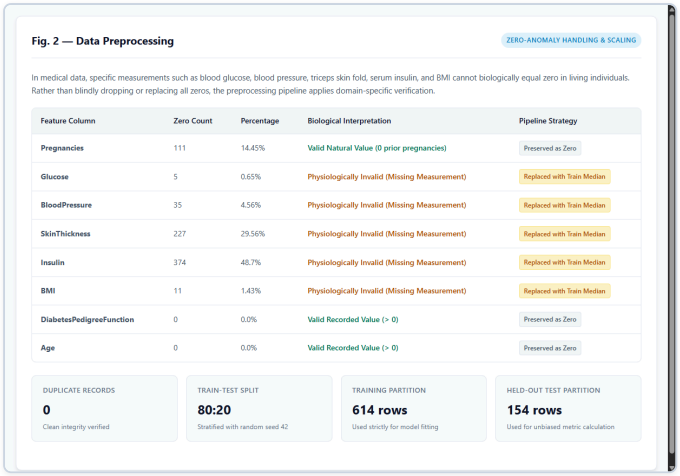


Fig. 2 — Data Preprocessing

ZERO-ANOMALY HANDLING & SCALING

In medical data, specific measurements such as blood glucose, blood pressure, triceps skin fold, serum insulin, and BMI cannot biologically equal zero in living individuals. Rather than blindly dropping or replacing all zeros, the preprocessing pipeline applies domain-specific verification.

Feature Column	Zero Count	Percentage	Biological Interpretation	Pipeline Strategy
Pregnancies	111	14.45%	Valid Natural Value (0 prior pregnancies)	Preserved as Zero
Glucose	5	0.65%	Physiologically Invalid (Missing Measurement)	Replaced with Train Median
BloodPressure	35	4.56%	Physiologically Invalid (Missing Measurement)	Replaced with Train Median
SkinThickness	227	29.56%	Physiologically Invalid (Missing Measurement)	Replaced with Train Median
Insulin	374	48.7%	Physiologically Invalid (Missing Measurement)	Replaced with Train Median
BMI	11	1.43%	Physiologically Invalid (Missing Measurement)	Replaced with Train Median
DiabetesPedigreeFunction	0	0.0%	Valid Recorded Value (> 0)	Preserved as Zero
Age	0	0.0%	Valid Recorded Value (> 0)	Preserved as Zero

DUPLICATE RECORDS	TRAIN-TEST SPLIT	TRAINING PARTITION	HELD-OUT TEST PARTITION
0	80:20	614 rows	154 rows
Clean integrity verified	Stratified with random seed 42	Used strictly for model fitting	Used for unbiased metric calculation

Fig 3: Step 2 — Data Preprocessing: missing/invalid zero analysis and stratified train-test split details

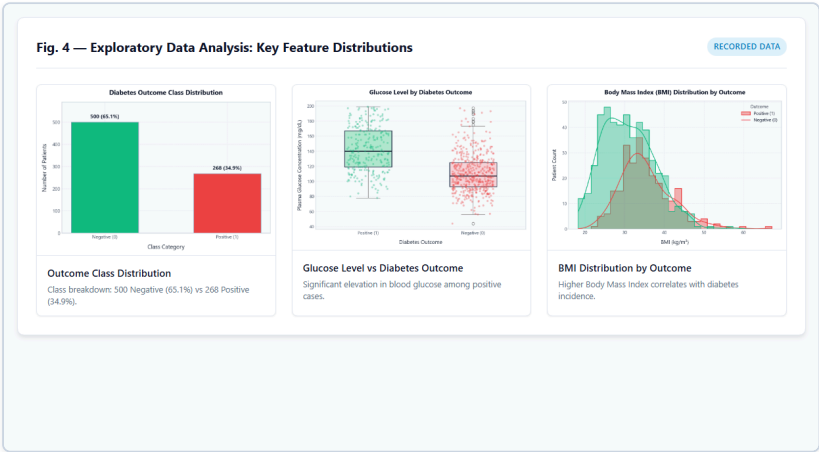


Fig 4: Step 3 — Exploratory Data Analysis: Outcome distribution, Glucose vs Outcome, and BMI distribution

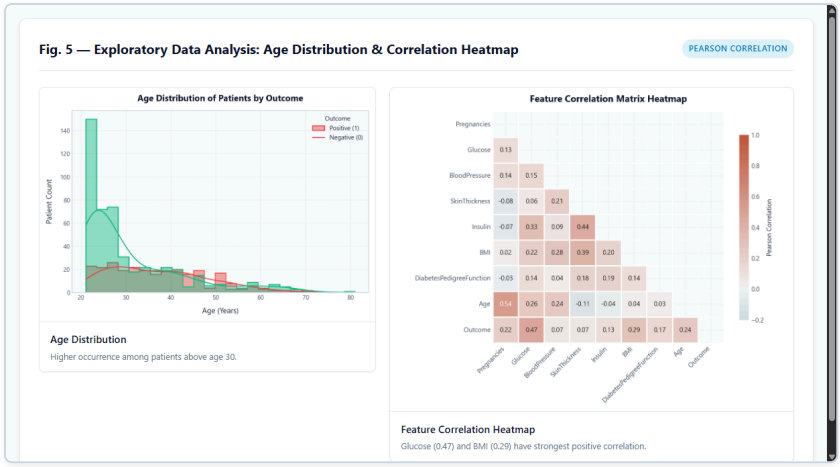


Fig 5: Step 3 — Exploratory Data Analysis: Age distribution and feature correlation matrix heatmap



Fig 6: Step 4 — Model Training & Comparison: multi-algorithm evaluation table and comparative bar chart

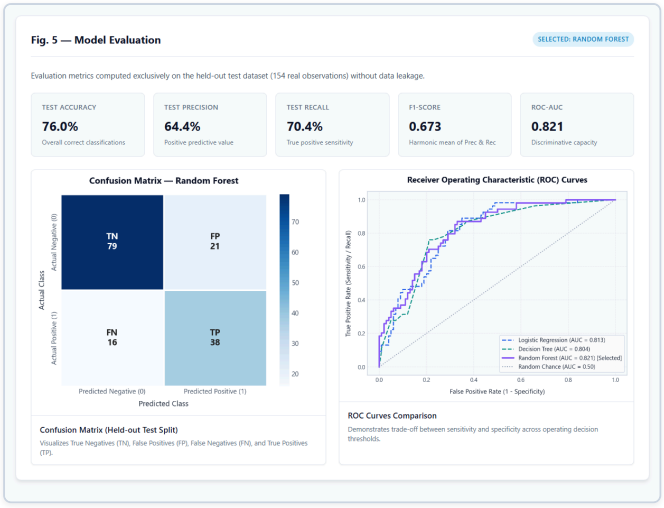


Fig 7: Step 5 — Selected Model Evaluation: held-out test metrics (Accuracy 75.97%, Precision 64.41%, Recall 70.37%, F1 0.6726, ROC-AUC 0.8213), confusion matrix and ROC curves

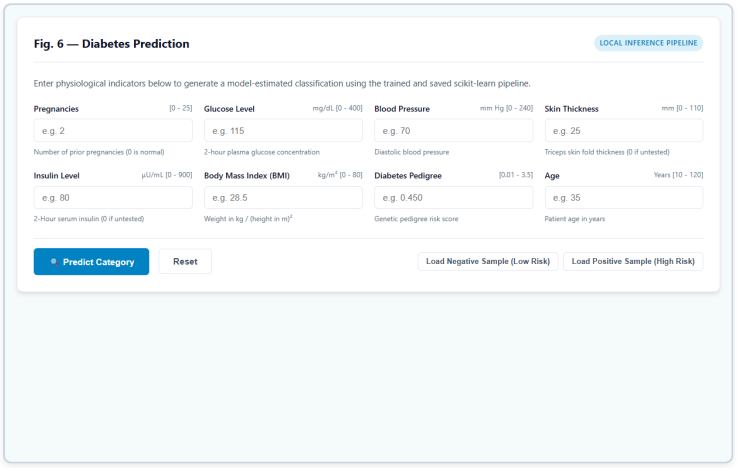


Fig 8: Step 6 — Diabetes Prediction Form: input parameter interface with physiological units and sample prefill buttons

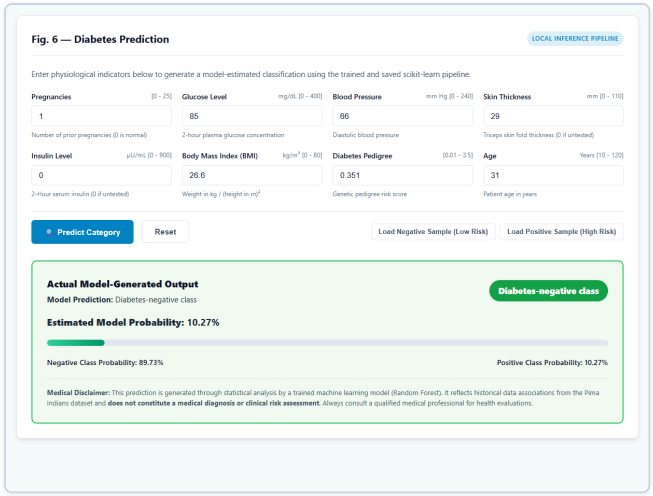


Fig 9: Step 6 — Classification Result: Diabetes-negative class (Healthy/Low-Risk), 89.7% negative probability (10.27% risk)

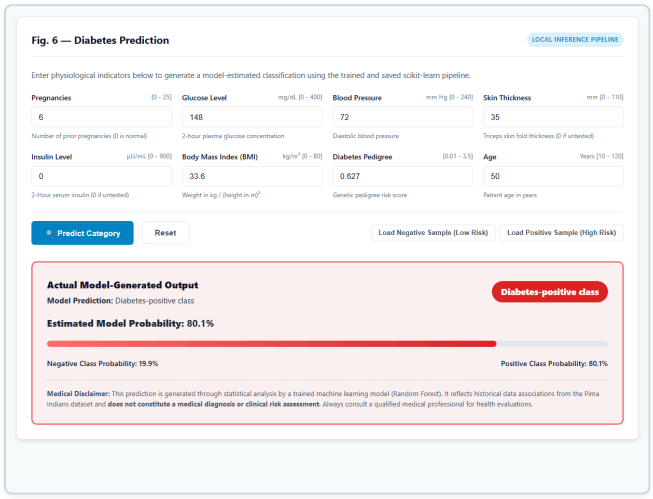


Fig 10: Step 6 — Classification Result: Diabetes-positive class (Elevated Risk), 80.1% positive probability

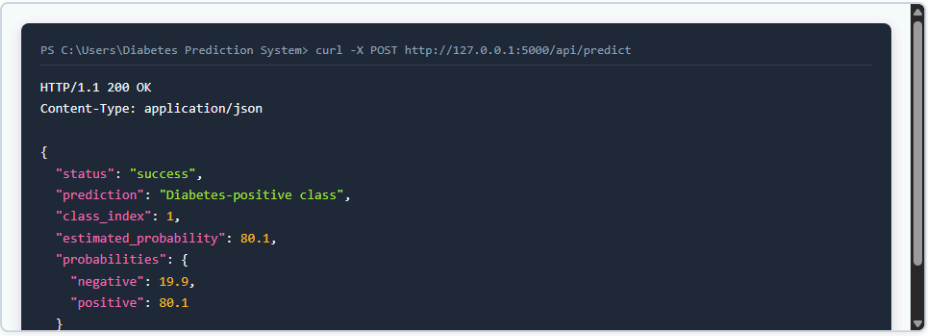


Fig 11: Step 7 — Verification of Local REST API (/api/predict) and JSON inference response

Result:

A complete, working web-based Diabetes Prediction System was successfully designed and implemented using a Flask backend, a Random Forest classification pipeline (scikit-learn), and an interactive light-themed web interface. Authentic clinical measurements from the Pima Indians Diabetes Database were preprocessed using domain-aware median imputation on physiologically invalid zero measurements (Glucose, Blood Pressure, Skin Thickness, Insulin, and BMI) while preserving valid natural zeros (Pregnancies), followed by standard feature scaling with strict elimination of data leakage.

Three classification algorithms (Logistic Regression, Decision Tree Classifier, and Random Forest Classifier) were trained and compared on an identical stratified 80:20 split. The Random Forest model demonstrated superior discriminative capability, achieving an **Accuracy of 75.97%**, **Precision of 64.41%**, **Recall of 70.37%**, an **F1-Score of 0.6726**, and an **ROC-AUC of 0.8213** on the held-out test set of 154 observations. The system validates real-time user inputs, computes predicted classification categories with estimated model probabilities, and provides a programmatic JSON REST API alongside clear educational and non-diagnostic medical disclaimers.